รายงานผลการทดสอบ

My First REST API — "Coffee Menu Service"

CP353002 — Principles of Software Design and Development

รายวิชา	CP353002 — Principles of Software Design and Development
ประเภทงาน	Mini Project — My First REST API ("Coffee Menu Service")
เทคโนโลยีที่ใช้	Java 17, Spring Boot 3.2.5, Maven
การเก็บข้อมูล	In-memory (List) — ยังไม่ใช้ฐานข้อมูล

1. ภาพรวมโปรเจกต์

โปรเจกต์นี้เป็นการสร้าง REST API เบื้องต้นด้วย Spring Boot สำหรับจัดการเมนูร้านกาแฟ (เพิ่ม / ดู / แก้ไข / ลบ) โดยออกแบบโครงสร้างโค้ดแบบแยกชั้น (Layered Design) ออกเป็น 3 ส่วนหลัก ดังนี้

- Model (Coffee) — เก็บโครงสร้างข้อมูลเมนูกาแฟ ได้แก่ id, name, price
- Service (CoffeeService) — จัดการ Business Logic และเก็บข้อมูลใน List แบบ in-memory
- Controller (CoffeeController) — รับ HTTP Request และเรียกใช้งานผ่าน Service เท่านั้น

ระบบรองรับ Endpoint หลักทั้งหมด 5 รายการ ได้แก่ GET /coffees, GET /coffees/{id}, POST /coffees, PUT /coffees/{id} และ DELETE /coffees/{id}

2. ผลการทดสอบ (Testing Results)

2.1 การรันแอปพลิเคชัน (mvn spring-boot:run)

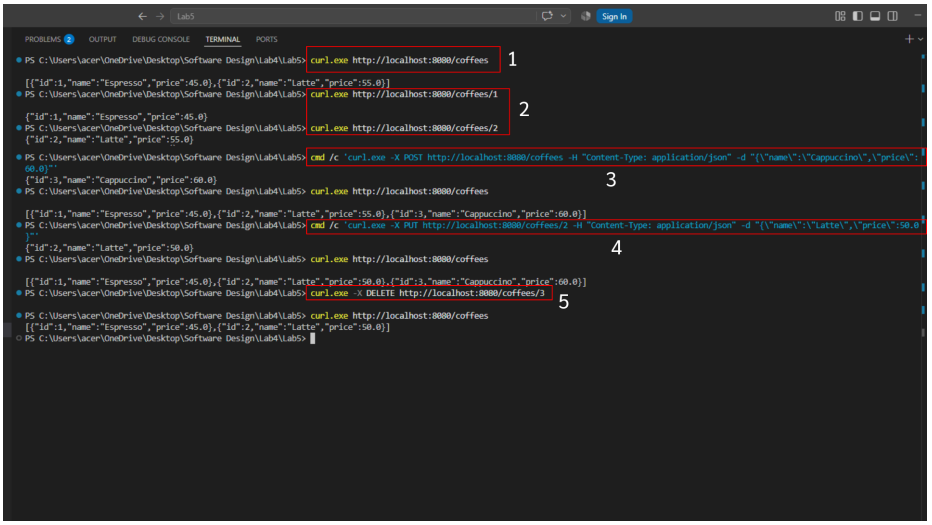
เริ่มต้นรันแอปพลิเคชันด้วยคำสั่ง mvn spring-boot:run ระบบสามารถเริ่มทำงานได้สำเร็จ และพร้อมรับ Request ที่พอร์ต 8080



ภาพที่ 1: ผลการรันแอปพลิเคชันด้วย mvn spring-boot:run

2.2 ผลการทดสอบ Endpoint ทั้ง 5 รายการ

ทดสอบ Endpoint ทั้งหมดผ่าน terminal (curl) ครบทั้ง 5 รายการ ได้แก่ GET /coffees, GET /coffees/{id}, POST /coffees, PUT /coffees/{id} และ DELETE /coffees/{id} ผลลัพธ์ที่ได้ตรงตามที่คาดหวังทุกกรณี



ภาพที่ 2: ผลการทดสอบ Endpoint ทั้ง 5 รายการ

3. คำถามท้ายบท (Discussion)

3.1 HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร

- GET: ใช้สำหรับดึง/เรียกดูข้อมูลโดยไม่มีการเปลี่ยนแปลงข้อมูลในระบบ เช่น GET /coffees เพื่อดูรายการเมนูทั้งหมด
- POST: ใช้สำหรับสร้างข้อมูลใหม่ขึ้นมาในระบบ เช่น POST /coffees พร้อมแนบ Body JSON เพื่อเพิ่มเมนูกาแฟใหม่
- PUT: ใช้สำหรับแก้ไข/อัปเดตข้อมูลเดิมที่มีอยู่ตาม ID เช่น PUT /coffees/2 เพื่อปรับเปลี่ยนราคาของ Latte
- DELETE: ใช้สำหรับลบข้อมูลออกจากระบบตาม ID เช่น DELETE /coffees/3 เพื่อลบรายการเมนูกาแฟออกจาก List

3.2 ทำไมต้องแยก Controller กับ Service ออกจากกัน

การแยก Controller กับ Service ยึดหลัก Separation of Concerns โดย Controller ทำหน้าที่จัดการ HTTP Request/Response และแปลง JSON ส่วน Service ทำหน้าที่จัดการ Business Logic และข้อมูล การแยกชั้นเช่นนี้ช่วยให้โค้ดเป็นระเบียบ อ่านง่าย และหากในอนาคตต้องการเปลี่ยนระบบจัดเก็บข้อมูลจาก Memory List เป็น Database ก็สามารถแก้ไขได้เฉพาะในชั้น Service โดยไม่กระทบ Controller

3.3 ข้อมูลใน List (in-memory) หายไปตอนไหน และควรทำอย่างไรไม่ให้หาย

- ข้อมูลใน List จะหายไปเมื่อแอปพลิเคชันหรือเซิร์ฟเวอร์หยุดทำงาน (Stop/Restart) เนื่องจากข้อมูลถูกเก็บไว้บน RAM (In-memory)
- หากต้องการให้ข้อมูลไม่หาย จะต้องเปลี่ยนไปใช้ระบบจัดเก็บข้อมูลแบบถาวร เช่น เชื่อมต่อฐานข้อมูล Relational Database (MySQL, PostgreSQL) หรือ NoSQL

3.4 หน้าที่ของ @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody

- @RestController: ระบุว่าคลาสนี้เป็น REST Controller ที่จะส่งค่ากลับไปเป็นข้อมูล (เช่น JSON) โดยตรงใน HTTP Response Body
- @GetMapping: แมป HTTP GET Request เข้ากับเมธอดที่กำหนด เพื่อใช้ดึงข้อมูล
- @PostMapping: แมป HTTP POST Request เข้ากับเมธอดที่กำหนด เพื่อใช้สร้างข้อมูลใหม่
- @PathVariable: ดึงค่าจาก Parameter บน URL Path มาใส่ในตัวแปรของ Java (เช่น ค่า {id} จาก /coffees/1)
- @RequestBody: แปลงข้อมูล JSON ที่ส่งมาใน HTTP Request Body ให้กลายเป็น Java Object โดยอัตโนมัติ

4. สรุป

โปรเจกต์ Coffee Menu Service สามารถพัฒนา REST API ครบทั้ง 5 Endpoint ตามที่กำหนด โดยออกแบบโครงสร้างโค้ดแบบแยกชั้น Model / Service / Controller อย่างชัดเจน ผลการทดสอบเป็นไปตามที่คาดหวังในทุกกรณี ทั้งการดึงข้อมูล เพิ่มข้อมูล แก้ไขข้อมูล และลบข้อมูล